OKEAEXPRESS

Integration discovery · for the partner's engineering team

How the OKEA EXPRESS platform works, what an integration must guarantee, and the questions we need answered

Please read sections 1–4 first: they describe the platform your system would have to talk to. Section 5 lists the questions. Your answers decide which integration shape is realistic.

FROM
OKEA EXPRESS · platform team

TO
Partner engineering & operations

VERSION
1.0 · 28 October 2026

1. Purpose

OKEA EXPRESS runs cash-on-delivery e-commerce logistics across Africa: order confirmation by phone and WhatsApp, warehousing, last-mile delivery by our own couriers, cash collection, returns, stock, inbound shipments, sourcing, seller accounts and finance — on one platform with its own phone apps. Three ways of working together are on the table:

Shape	What it means	Integration needed
A · Adopt the OKEA platform (recommended)	Your company becomes a tenant on the platform with its own isolated data, users and branding. Your staff use the OKEA apps (manager workspace, confirmation app, warehouse / cashier app, courier app).	None. Configuration, training and support.
B · Carrier integration	OKEA keeps confirmation, warehouse, stock, sourcing, sellers and finance. Your system performs the last-mile delivery only: OKEA pushes shipments to you, you push delivery events, proof and collected cash back.	A narrow, event-driven carrier API (section 4).
C · Full two-way mirror (not recommended)	Both platforms stay in full use and every action is replicated in both directions.	A complete bidirectional sync of orders, parcels, couriers, cash, stock and users. Two sources of truth.

We do not consider shape C viable (two systems of record for the same order, conflicting state, and most OKEA features depend on data your system does not produce). This brief is written so that shape A or B can be assessed honestly from your answers.

2. The OKEA EXPRESS platform in one page

2.1 Modules of the management workspace

Module	What it does
Dashboard	Live KPIs per country and period: orders, confirmation rate, delivery rate, cash collected, returns, stock alerts.
Confirmation	Orders (from the online store, manual entry, Excel import, webhooks), a fair queue split between agents, call outcomes, callbacks, discounts with approval, duplicate / spam handling, address editing, a follow-up stage for postponed orders, and the **WhatsApp Agent** (automatic WhatsApp after an unanswered call, with confirm / edit buttons).
Delivery	Shippings by status, **Assign to courier** by scanning the parcel's QR label, returns (expected, scanned back, graded, restocked or re-delivered), **cash handover** (per-courier cash-ups counted by the cashier), couriers & **live map** (GPS positions and trails from the courier app).
Warehouse	Parcels to prepare, square branded labels with QR (100×100, 80, 75 mm), shelves / bins with capacity, pick-pack-stage, scan stations with sound and vibration.
Stock	SKUs with available / reserved quantities per bin and hub, corrections with second-person approval, low-stock alerts, reorder points.
Products · Shipments · Sourcing	Product catalogue linked to SKUs; inbound shipments (expected → arrived → counted → put away, with discrepancy handling and photos); sourcing requests from suppliers with quotes, deposits, purchase orders, transit tracking and landed cost.
Finance	Overview per currency, reconciliation, cashier invoices (generate, validate, pay with proof), courier pay runs, seller balances and payouts.
Network · Sellers · Team	All 54 African countries and currencies; hubs and zones (including far zones); seller onboarding with KYC and isolated seller workspaces; staff with roles, scopes (country, hub, zones) and separation-of-duties rules; internal chat.
Store	A cash-on-delivery online store (product pages, checkout, pixels) whose orders flow straight into confirmation.

2.2 Phone apps

- **OKEA app** — managers and confirmation agents (orders arrive by themselves, one-tap outcomes, chat).
- **OKEA Hub** — warehouse and cashier: camera scanning, labels, cash-ups, returns.
- **Courier app** — mandatory location and notifications; doorstep flow *scan QR → Delivered (customer code or hand-over photo) / Reprogram / Cancel*; failed-delivery proof (photo + customer voice note); cash drop at the hub.
- **Store app** — the store back office on the phone.

Android APK and iPhone installation exist for every app; all run full-screen with a branded notification sound.

2.3 The order and parcel lifecycle (states you would have to map)

NEW → **PENDING (to call)** → **CONFIRMED** / **POSTPONED** / **OUT_OF_STOCK** / **CANCELLED** / **DUPLICATE · SPAM**

RESERVED → **TO_PREPARE** → **PREPARED (labelled)** → **ASSIGNED (scanned out to a courier)** → **SHIPPED** → **DELIVERED** / **FAILED → RETRY / RETURN_EXPECTED** → **RETURNED → GRADED → RESTOCKED / RELAUNCHED** → **PAID (cash counted)**

Every transition is an event with an actor (staff id, courier id, seller member, system), a timestamp, the device where relevant, and an audit row. Money is tracked per parcel (COD amount, discounts, delivery charge) and per courier cash-up; stock is reserved at confirmation and released or consumed by the parcel's outcome.

2.4 Core entities

Entity	Key attributes
Country / Hub / Zone	ISO code, currency, phone rules; hub = warehouse with bins; zone = delivery area with far-zone rules.
Seller	The merchant whose goods are sold (OKEA's own company is one seller; partners are others). All data is scoped by seller.
Order	Order number, customer (name, phone E.164, address, zone), lines (SKU, qty, price), COD amount, source, confirmation state and history.
Parcel	Code (QR), order, hub, courier, sequence, label prints, delivery attempts, proof, cash.
Courier	Staff member with vehicle, zones, device, live position, cash balance, pay run.
SKU / Bin	Product variant with availability per bin; movements, counts, corrections.
Cash-up / Invoice / Payout	Cash per courier per day; cashier invoices; seller balances and payouts.

2.5 Technical facts about our side

- Modern web platform (TypeScript, PostgreSQL) with role-based access control, per-tenant data isolation enforced in the database, full audit log, and signed sessions per surface.
- Multi-country and multi-currency by design; every amount stores its currency and FX rate at write time.

- Existing outbound integrations: webhooks for leads, Google Sheets, Meta Conversions API, WhatsApp Cloud API, web push. Inbound: order webhooks, Excel import.
- We expose and consume REST + JSON with idempotency keys and HMAC-signed webhooks.

3. What shape A needs from you (no API)

- A list of your hubs, zones and couriers (name, phone, vehicle, zones).
- Your staff list with the role each person will hold (manager, confirmation agent, warehouse, cashier, dispatcher, courier).
- Your product list with SKUs and current stock per hub (Excel is fine).
- Optional: your logo and colours for a white-labelled workspace and apps.
- Half a day of training per role; a two-week pilot in one city is the usual start.

4. What shape B (carrier integration) must guarantee

If delivery runs in your system, this is the minimum contract. Items marked `must` are non-negotiable for OKEA to operate.

Requirement	Detail
Shipment creation must	OKEA pushes a shipment (our parcel code, customer name, phone, address, zone / GPS, COD amount and currency, number of pieces, product summary, delivery window, our label PDF). Your API returns your tracking id synchronously. Idempotent on our parcel code.
Status events must	Webhooks to OKEA within 60 seconds for: accepted, picked up, at hub, out for delivery, delivered, failed (with reason code), rescheduled (with date), cancelled, return started, returned to OKEA. Each event carries our parcel code, your event id, timestamp (UTC), courier id and GPS where available. Signed (HMAC) and retried with backoff; we answer 2xx and de-duplicate on your event id.
Proof of delivery must	Delivered events include at least one of: the customer's delivery code (we give it to you at creation), a hand-over photo, or a signed receipt. Failed events include a reason and, ideally, a photo or voice note.
Cash collected must	The exact amount collected per parcel (not a daily total), the courier who collected it, and a daily settlement statement per hub that reconciles to the sum of delivered parcels. Remittance to OKEA on an agreed schedule.
Cancellations and changes must	OKEA can cancel or change the address / phone / COD amount until the parcel is out for delivery; your API confirms or refuses with a reason.
Returns	Returned parcels come back to an OKEA hub with your tracking reference; a return event is sent when the parcel leaves your hub and when it is handed to OKEA.
Courier positions	Optional but valued: GPS positions of the courier carrying our parcels while out for delivery (for our live map and customer ETA).
Reference data	Your zones / cities with codes so we can map ours; service availability and cut-off times per zone.
Authentication & environments	OAuth2 client credentials or API keys per environment; a sandbox with test data; versioned API with a deprecation policy.
Reliability & support	Published uptime target, rate limits, status page, incident contact, and a reconciliation report (daily) comparing both sides.
Data protection	Customer data used only for delivery; deletion on request; no sharing with third parties; data location disclosed.

5. Questions for your team

Short answers are fine. "Not available" is a valid and useful answer. Where you have documentation, a link replaces the answer.

5.1 Architecture and ownership

A1 Who builds and maintains your platform (in-house team, agency, vendor product)? Who would own the integration work and its ongoing maintenance?

Why: an integration needs a counterpart that can change code and answer incidents.

A2 Is your platform a single-tenant system for your company, or a product used by several companies? Can it isolate a client's data (OKEA's orders) from others?

A3 Which stack and hosting (country / provider) do you use? Where is customer data stored?

A4 Do you have a public or partner API today? Please share the documentation, authentication method and versioning policy.

A5 Do you support outbound webhooks (you call us when something happens)? If not, what polling interval would be acceptable, and what is the rate limit?

A6 Do you have a sandbox / staging environment with test data we can use?

A7 How do you handle idempotency (duplicate requests) and ordering of events?

5.2 Orders and confirmation

B1 Where do orders enter your system today (store, Excel, API, manual)? Which fields does an order carry?

B2 Do you run order confirmation (calling the customer) inside your system? If yes: queue logic, call outcomes, callbacks, postponed orders, WhatsApp automation?

B3 List your order and parcel statuses with their exact meaning and the allowed transitions.

B4 Can an order be changed (address, phone, COD amount, lines) after creation, and until which status?

B5 How are duplicates, spam and out-of-stock orders handled?

5.3 Delivery and couriers

C1 Do your couriers use a mobile app? Which platform (Android / iOS / web)? What does the courier do on the app at pickup, at the door, and after a failed attempt?

C2 Is GPS tracking of couriers recorded? At what frequency? Is it available by API?

C3 How is a parcel assigned to a courier (manual, scan, automatic by zone)? Are parcels labelled with a barcode or QR, and whose label is used?

C4 What proof of delivery do you capture (customer code, signature, photo, OTP)? Is it stored and retrievable by API?

C5 What happens on a failed delivery: reasons list, retry rules, maximum attempts, who decides the return?

C6 How do you model service areas (cities, zones, polygons)? Can you share the list with codes? Any far-zone or surcharge rules?

C7 What are your cut-off times, delivery windows and typical delivery lead time per zone?

5.4 Cash and finance

D1 Is cash recorded per parcel at the moment of delivery, or as a courier's daily total? Who counts it and where?

D2 How and when is cash settled to the merchant (schedule, method, fees, currency)? Can a statement be exported or fetched by API?

D3 How do you handle discounts given at the door, partial payments, and customer refusals?

D4 How are couriers paid (per parcel, salary, mixed) and does your system compute it?

5.5 Warehouse, stock and inbound

E1 Do you operate warehouses for merchant stock? Is stock tracked per SKU and location (bin / shelf)? Reserved vs available?

E2 How do returns re-enter stock (grading, damaged goods, re-delivery to a new customer)?

E3 Do you receive inbound shipments from merchants or suppliers in the system (expected quantities, counting, discrepancies)?

5.6 Products and merchants

F1 Does your system have the concept of a merchant / seller with isolated data and its own login? Can a merchant see only their own orders, stock and balance?

F2 How are products and SKUs modelled (variants, barcodes, pricing per country)?

5.7 Users, roles and security

G1 Which roles exist (admin, dispatcher, agent, warehouse, cashier, courier…) and what can each do? Can permissions be scoped by hub or zone?

G2 Is there an audit log of who did what and when? Can it be exported?

G3 Authentication for staff and couriers (password, OTP, device binding)? Session lifetime? Two-factor for admins?

G4 Backups, retention and disaster recovery: how often, where, tested when?

5.8 Operations and service levels

H1 Current volumes: parcels per day per city, number of couriers, number of hubs. Peak volumes you have handled.

H2 Uptime over the last 12 months, incident process, and who OKEA contacts when something breaks at 9 pm.

H3 What reporting exists (daily delivery rate, return rate, cash reconciliation) and can it be fetched by API?

H4 Can you provide a daily reconciliation file (parcel code, status, cash) that OKEA can compare against its own records automatically?

5.9 Commercial and project

I1 If an API integration is chosen: estimated effort and timeline on your side, who pays for the build and the maintenance, and how changes to your API are communicated.

I2 If your couriers used the OKEA courier app for a two-week pilot in one city, what would be your concerns? What would you need from OKEA to make it work (training, devices, branding, language)?

I3 Exit terms: if the partnership ends, how is OKEA's data (orders, customers, proofs, statements) exported and deleted?

6. Next steps

1. Your written answers to section 5 (a shared document is fine) within two weeks.
2. A one-hour technical call to walk through your answers and a live demo of the OKEA apps.
3. Decision: shape A with a pilot, or shape B with the carrier contract in section 4 as the specification.

Our position in one sentence

OKEA EXPRESS is a complete way of working, not an API: we recommend running it as is — with your team, your data and your branding — and we keep a narrow carrier integration as the fallback. A full two-way mirror of two platforms is not something we will build.